

BitcoinPipeline

Rapport E1 : Mise en situation — Collecter, stocker et mettre à disposition des données

Champ	Détail
Candidat	Sullivan Moreau (Satoshi)
Diplôme	Développeur en Intelligence Artificielle, Simplon, promotion 2026
Compétences évaluées	C1, C2, C3, C4, C5
Commanditaire	Sans objet, projet individuel mené dans le cadre de la certification
Dépôt Git	github.com/sullivan-moreau/BitcoinPipeline
Date de rédaction	20 juillet 2026
Soutenance	31 juillet 2026

Contexte du projet

BitcoinPipeline collecte, agrège et expose le prix du Bitcoin depuis cinq sources hétérogènes. Le projet répond à une question métier : quel est le prix du Bitcoin selon différentes méthodes de collecte (service web, scraping, fichier, base de données, big data), et comment le rendre disponible de façon fiable et sécurisée via une API ?

Composant	Technologie	Rôle
Pipeline d'extraction	Python 3.12 / uv	Collecte, agrégation, persistance
Base de données	PostgreSQL 16 (bitcoin_db)	Stockage du jeu de données final
Big Data	Apache Spark 3.5 (PySpark)	Archive Bitfinex (~4,5 M lignes, Parquet)
Scraping	requests + BeautifulSoup4	Page marchés CoinLore (exchange Kraken)
API	FastAPI + JWT (Argon2)	Exposition REST sécurisée
Orchestration	Docker Compose	Reproductibilité de PostgreSQL

C1 — Cinq sources, cinq méthodes d'extraction

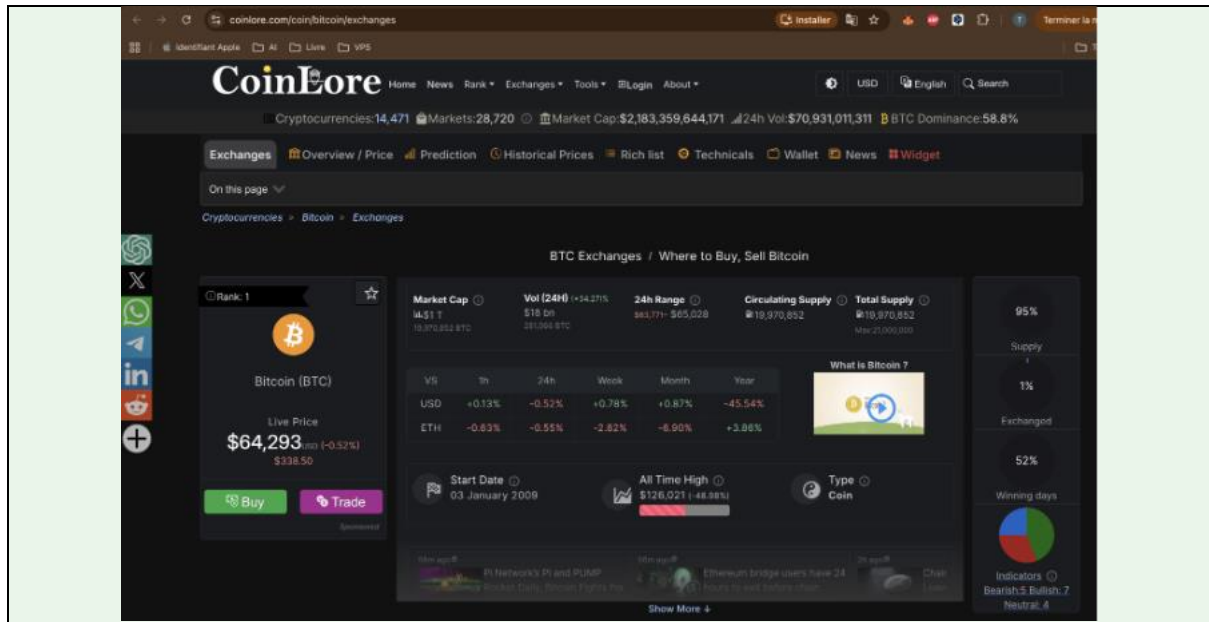
Source	Extracteur	Méthode
CoinGecko	api_collector.py	API REST
Kraken (via page marchés CoinLore)	scraper_collector.py	Scraping HTML (BeautifulSoup)
Coinbase	file_collector.py	Lecture de fichier CSV (dataset Kaggle)
Bitstamp	db_collector.py	Requêtes SQL sur PostgreSQL
Bitfinex	bigdata_collector.py	Apache Spark (archive Parquet)

Le périmètre couvre l'ensemble des moyens d'extraction requis : API REST, scraping, fichier, base de données et big data. Chaque extracteur expose une fonction `run()` avec un point de lancement unique (`if __name__ == "__main__":`) et une gestion des erreurs systématique : chaque défaillance (timeout réseau, page indisponible, fichier absent, base inaccessible) est journalisée puis traitée sans interrompre les autres sources.

La page kraken.com bloque activement le scraping automatisé (403 confirmé en test). La solution retenue scrape la page de marchés publique de CoinLore, qui liste le prix Kraken parmi plusieurs exchanges, avec priorisation des paires indexées sur le dollar (USDT puis USDC).

```
uv run python -m src.extract.api_collector
uv run python -m src.extract.scrapers_collector
uv run python -m src.extract.file_collector
uv run python -m src.extract.db_collector
uv run python -m src.extract.bigdata_collector
```

Scripts versionnés sur github.com/sullivan-moreau/BitcoinPipeline.



C2 — Requêtes SQL documentées et optimisées

Requêtes documentées dans [docs/sql_queries.md](#), avec résultats réels obtenus en conditions de test.

Requête 1 — Dernier prix Bitstamp (SGBD)

```
SELECT ts_unix, close
FROM historical_prices_bitstamp
ORDER BY ts_unix DESC
LIMIT 1
```

Le tri décroissant sur le timestamp Unix fait remonter l'enregistrement le plus récent ; LIMIT délègue le filtrage au moteur SQL plutôt qu'à un tri applicatif.

Requête 2 — Dernier prix Bitfinex (Big Data)

```
df.orderBy(col("time").desc()).first()
```

Équivalent logique à un ORDER BY... LIMIT 1 SQL. `orderBy(desc)` suivi de `first()` permet à Spark d'optimiser l'opération en un partitionnement distribué, sans jamais charger l'intégralité des 4,5 millions de lignes en mémoire driver.

Optimisations : LIMIT délégué au moteur plutôt qu'un tri côté application, lecture colonnaire du Parquet (seules les colonnes time et close sont lues) plutôt qu'une lecture CSV complète.

J'utilise SQLAlchemy Core (text(), SQL brut paramétré) plutôt que l'ORM déclaratif, pour garder le contrôle explicite sur les requêtes exécutées.

C3 — Agrégation multi-sources

Les cinq sources retournent déjà une même grandeur, un prix en dollars, ce qui simplifie l'agrégation à un problème de format plutôt que de sens. Le script `src/normalize.py` applique :

1. Collecte isolée : chaque collecteur est appelé dans un bloc try/except indépendant, l'échec d'une source ne bloque jamais les quatre autres.
2. Validation : rejet des entrées dont une clé manque, dont le prix sort de la plage 1 à 1 000 000 USD, ou dont les timestamps ne sont pas des dates ISO 8601 valides.
3. Fusion et homogénéisation via un DataFrame pandas : arrondi du prix à 2 décimales, timestamps reformatés en ISO 8601 strict (suffixe Z).
4. Tri final par ordre alphabétique de source, pour une sortie stable et reproductible.

L'agrégation est réalisée avec pandas plutôt qu'en SQL, plus adapté pour combiner des formats très différents (JSON, HTML parsé, CSV, résultats SQL et PySpark) avant l'écriture dans un format commun.

Le référentiel demande d'agréger au moins deux sources en SQL, via un JOIN ou une concaténation de tables. Ce point est couvert par une jointure dédiée entre Bitstamp et Coinbase (`src/aggregate_sql.py`, `docs/sql_queries.md`, Requête 5), avec deux CTE (WITH) et un JOIN sur une fenêtre horaire commune. Coinbase est chargé au préalable dans une table Postgres dédiée via un script de seed, comme pour l'échantillon Bitstamp ; le JOIN s'exécute ensuite entre deux tables Postgres classiques.

Point d'attention assumé : les cinq sources n'ont pas la même fraîcheur temporelle. CoinGecko et le scraping Kraken sont en temps réel ; Coinbase, Bitstamp et Bitfinex sont des instantanés figés à la date de collecte de leur dataset (Bitfinex date d'octobre 2023, d'où un prix autour de 27 900 dollars contre environ 64 000 dollars pour les sources en temps réel). Cet écart n'est pas une anomalie : il reflète la nature de chaque méthode d'extraction, pas un comparateur de prix en direct.

```
source,price_usd,timestamp,collected_at
api_coingecko,63999.0,2026-07-20T09:04:24.400701Z,2026-07-20T09:04:24.400701Z
bigdata_bitfinex,27912.0,2023-10-08T09:28:00Z,2026-07-20T09:04:27.987411Z
db_bitstamp,64589.48,2026-07-16T01:35:00Z,2026-07-20T09:04:24.920650Z
file_coinbase,43312.27,2022-03-01T00:00:00Z,2026-07-20T09:04:24.881502Z
scraping_kraken,64031.0,2026-07-20T09:04:24.866999Z,2026-07-20T09:04:24.866999Z
```

C4 — Modélisation Merise et conformité RGPD

Le projet repose sur une unique instance PostgreSQL 16 (`bitcoin_db`), avec deux entités indépendantes : `bitcoin_prices` (jeu de données final) et `users` (comptes d'authentification de l'API). Modélisation complète (MCD et MPD, formalisme Merise) dans `docs/merise_mcd.md`.

PostgreSQL 16 retenu pour sa gestion native des contraintes et des upserts (ON CONFLICT), et son déploiement simple via Docker Compose.

Reproductibilité : vérifiée par un clone à blanc réel du dépôt (git clone, uv sync, puis import de `src.normalize`) : aucune erreur, l'ensemble des scripts est bien versionné et accessible depuis le dépôt Git.

Conformité RGPD

Les prix Bitcoin collectés sont des cours de marché publics, sans donnée personnelle. Le seul traitement de données personnelles du projet concerne le compte administrateur technique de l'API (table users), documenté dans docs/rgpd_procedures.md :

Champ	Valeur
Finalité	Sécuriser l'accès en lecture aux endpoints de l'API
Base légale	Intérêt légitime (sécurité du service)
Personnes concernées	Administrateur technique du projet (1 compte)
Données traitées	Username, hash Argon2, date de création
Conservation	Durée de vie du projet pédagogique

Procédures associées : suppression du compte sur demande, rotation du secret JWT en cas de doute sur une fuite, réinitialisation complète possible via docker compose down -v. Aucune automatisation : le volume traité (un seul compte) ne le justifie pas.

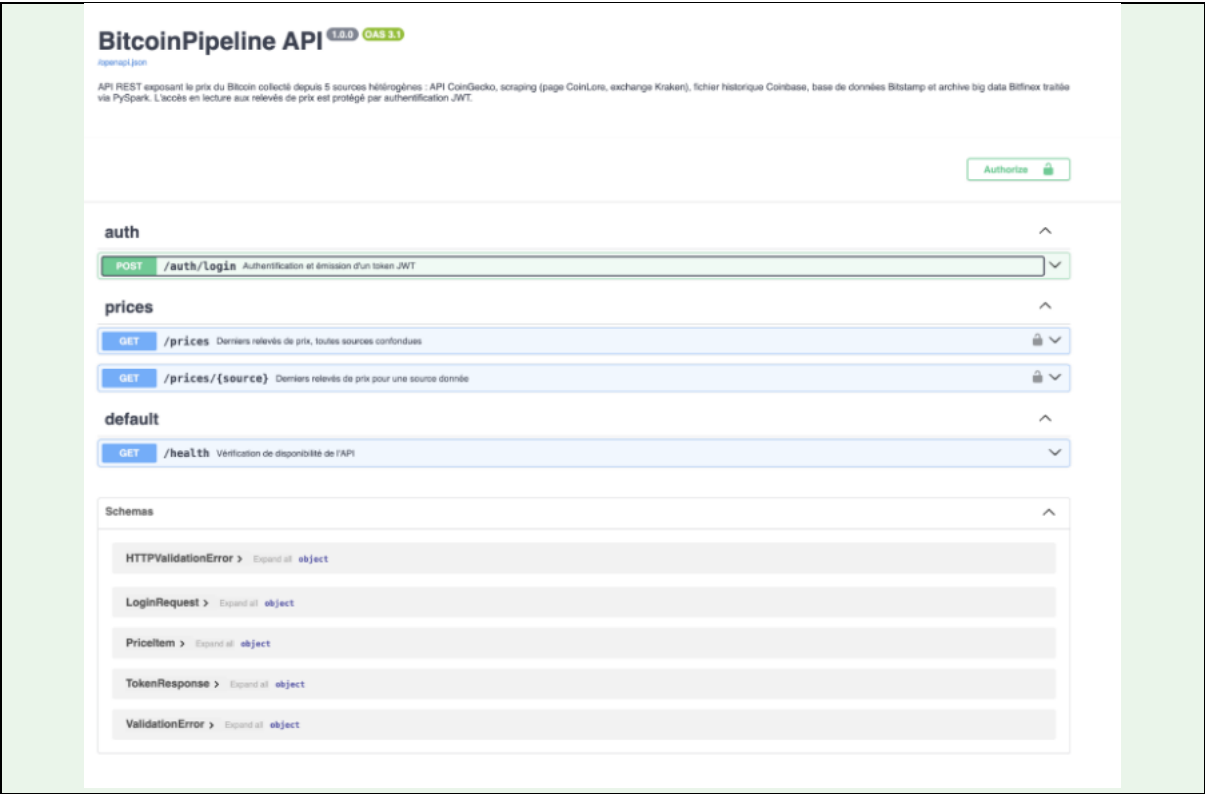
C5 — API REST, sécurité et synthèse

L'API FastAPI expose le jeu de données de prix Bitcoin, documentée dans docs/openapi.md (schéma OpenAPI 3.1, interface Swagger sur /docs).

Méthode	Route	Protégée	Description
POST	/auth/login	Non	Authentification, émission d'un token JWT
GET	/prices	Oui (JWT)	20 relevés les plus récents, toutes sources confondues
GET	/prices/{source}	Oui (JWT)	20 relevés les plus récents pour une source donnée
GET	/health	Non	Vérification de disponibilité

Toutes les routes de données sont protégées par un token JWT Bearer (HS256, python-jose), vérifié par la dépendance get_current_user(). Les mots de passe sont hashés en Argon2 (argon2-cffi).

Testé en conditions réelles le 20 juillet 2026 : authentification réussie retournant un token valide, échec 401 sur identifiants incorrects, accès refusé (401) sans token ou avec un token invalide, récupération effective des cinq relevés de prix avec un token valide.



Conclusion

BitcoinPipeline est aujourd’hui un pipeline complet et fonctionnel : cinq sources collectées, agrégées, stockées en PostgreSQL et exposées via une API sécurisée par JWT. Vérifié par exécution réelle le 20 juillet 2026.